MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 2 / 8

The Sigmoid Neuron: Logistic Regression

Replace the step by a smooth function, derive the loss and its gradient by hand, then code it in NumPy and PyTorch.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

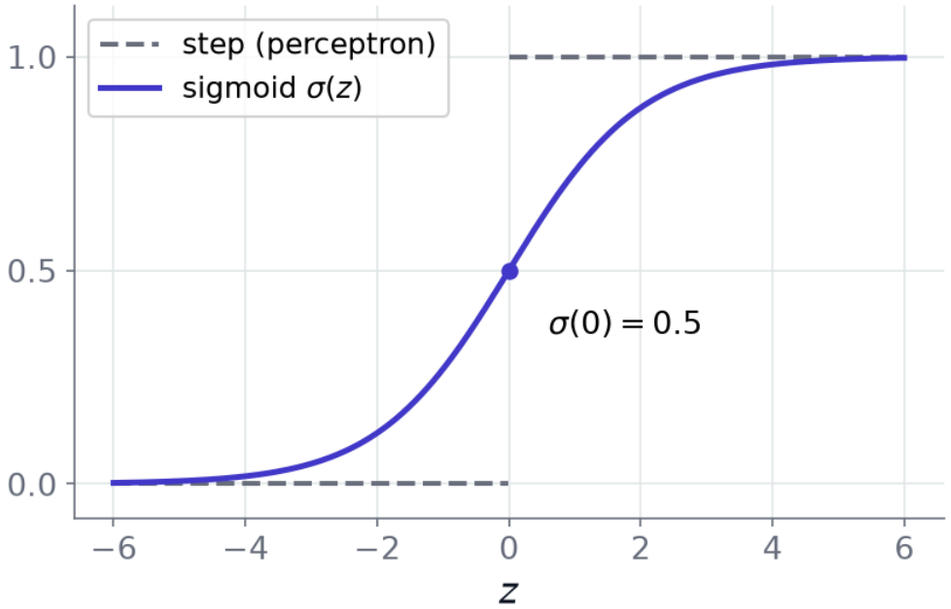
Last time: the limits of the perceptron

The perceptron outputs 0 or 1 with a step function. Two problems:

- no **confidence**: $z = 0.01$ and $z = 100$ both give 1;
- no **gradient**: the step is flat, so no gradient descent.

Today's fix: keep $z = w^\top x + b$ but replace the step by the **sigmoid**:

$$a = \sigma(z) = \frac{1}{1 + e^{-z}}$$



A smooth step: same idea, but differentiable.

Reading the sigmoid as a probability

Properties

- $0 < \sigma(z) < 1$, and $\sigma(0) = 0.5$.
- $z \to +\infty$: $\sigma \to 1$. $z \to -\infty$: $\sigma \to 0$.
- Symmetry: $\sigma(-z) = 1 - \sigma(z)$.

Interpretation

$$a = \sigma(w^\top x + b) = P(y = 1 \mid x)$$

This model is **logistic regression**: one neuron with a sigmoid activation.

Decision

Predict class 1 when $a \geq 0.5$:

$$a \geq 0.5 \iff \sigma(z) \geq \sigma(0) \iff z \geq 0$$

(the sigmoid is increasing). So the decision boundary is **still the line** $w^\top x + b = 0$.

What changed

Same line as the perceptron, but now each point gets a probability: far from the line, a is close to 0 or 1; near the line, $a \approx 0.5$.

Exercise 1 · The sigmoid and one forward pass

Exercise 8 min pen and paper, calculator

1. Compute $\sigma(0)$, $\sigma(2)$, $\sigma(-2)$ and $\sigma(4)$. Use $e^{-2} \approx 0.135$ and $e^{-4} \approx 0.018$. Check the symmetry $\sigma(-2) = 1 - \sigma(2)$.
2. A neuron has $w = (2, -1)$ and $b = -1$. For the example $x = (1, 3)$, compute z, then $a = \sigma(z)$.
3. What is $P(y = 1 \mid x)$? Which class is predicted?

Your notes / answers (saved in this browser)

How good is a prediction? The likelihood

The model says $P(y = 1 \mid x) = a$ and $P(y = 0 \mid x) = 1 - a$. Both cases in one formula (Bernoulli):

$$P(y \mid x) = a^{y}\,(1-a)^{1-y} \qquad \begin{cases} y = 1 : & a^{1}(1-a)^{0} = a \\ y = 0 : & a^{0}(1-a)^{1} = 1 - a \end{cases}$$

For a dataset of m independent examples, the **likelihood** is the product of these probabilities:

$$L(w, b) = \prod_{i=1}^{m} \left(a^{(i)}\right)^{y^{(i)}} \left(1 - a^{(i)}\right)^{1-y^{(i)}}$$

Good parameters give a high probability to the labels we observed: we want to **maximise** L.

From the likelihood to the log loss, step by step

1. Take the log (it is increasing: same maximum; and a product of m numbers below 1 underflows to 0):

$$\log L = \sum_{i=1}^{m} \log \left[\left(a^{(i)} \right)^{y^{(i)}} \left(1 - a^{(i)} \right)^{1-y^{(i)}} \right] = \sum_{i=1}^{m} \left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)}) \right]$$

2. Maximising $\log L$ is minimising $-\log L$. Divide by m to get a mean that does not depend on the size of the dataset:

$$J(w, b) = -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \log a^{(i)} + (1 - y^{(i)}) \log(1 - a^{(i)}) \right]$$

Vocabulary

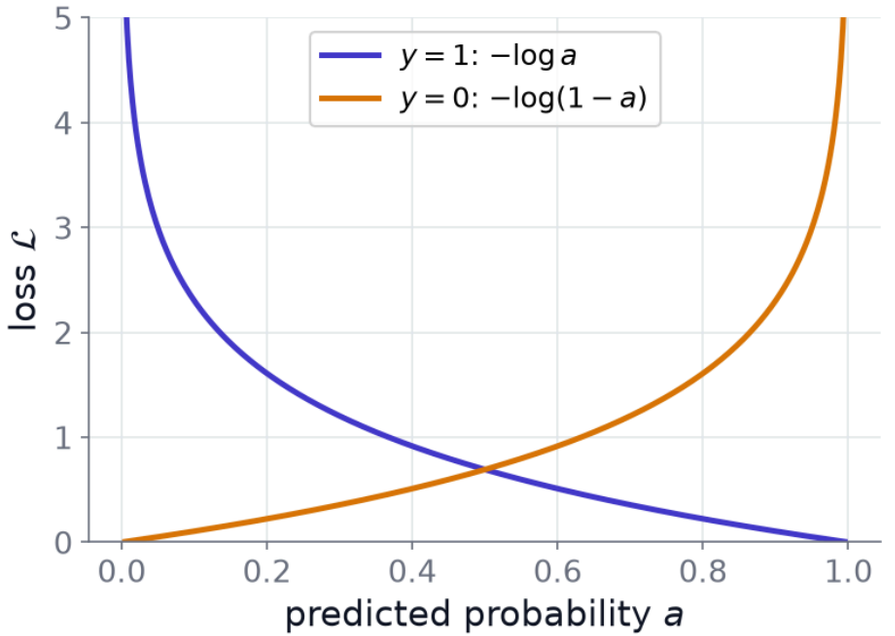
J is the **log loss**, or **binary cross-entropy**. Loss of one example: $\mathcal{L} = -[y \log a + (1 - y) \log(1 - a)]$; the cost J is the mean of $\mathcal{L}$ over the dataset.

Reading the log loss

Only one term is active for each example:

- $y = 1$: $\mathcal{L} = -\log a$. Small if $a \approx 1$, huge if $a \approx 0$.
- $y = 0$: $\mathcal{L} = -\log(1 - a)$. Small if $a \approx 0$, huge if $a \approx 1$.

A confident wrong prediction costs a lot ($-\log a \to \infty$ when $a \to 0$); a correct, confident one costs almost nothing.



Exercise 2 · Three log losses by hand

Three examples, all with true label $y = 1$. The model predicts $a = 0.9$, $a = 0.5$ and $a = 0.1$.

Exercise 7 min pen and paper, calculator

1. Compute the loss $\mathcal{L}$ of each example (natural log).
2. Compute the cost J of these 3 examples.
3. Which example contributes most to J? What happens to its loss if $a = 0.01$?

Your notes / answers (saved in this browser)

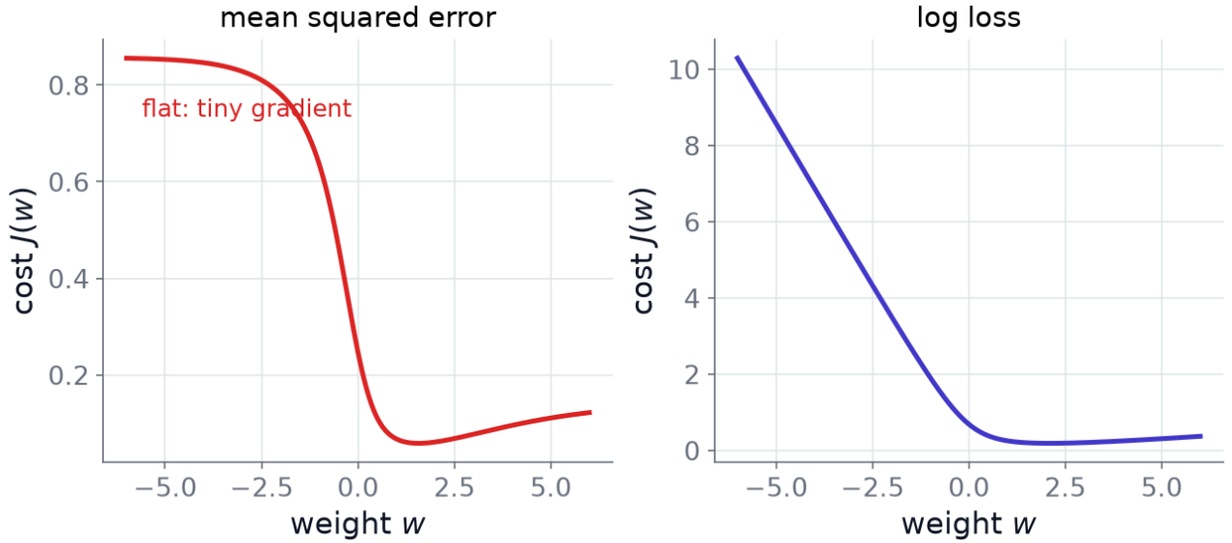
Why not the mean squared error?

We could minimise $\frac{1}{m}\sum(a-y)^2$. Two problems with a sigmoid output:

- **Not convex** in w: flat regions where gradient descent crawls.
- **Weak gradient**: $\frac{\partial}{\partial z}(a-y)^2 = 2(a-y)\,a(1-a)$. Very wrong ($a \approx 0$, $y = 1$): $a(1-a) \approx 0$, almost no correction.

The log loss instead

It is **convex** for logistic regression and, as we will see, its gradient is simply $(a-y)\,x$: no $a(1-a)$ factor to kill it.



Cost as a function of one weight on a small 1-D dataset.

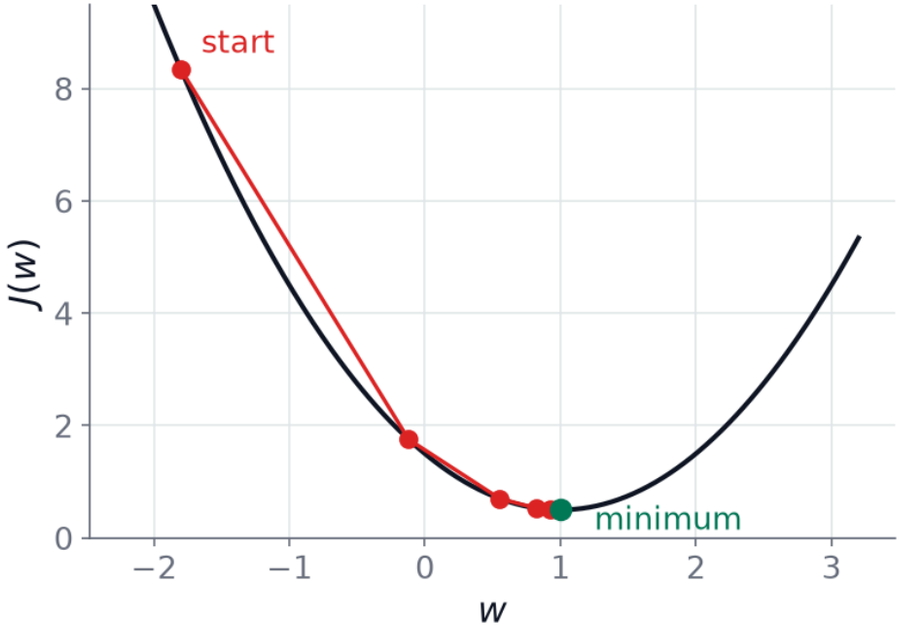
Gradient descent in one slide

To minimise $J(w, b)$, start somewhere and repeat a small step **downhill**:

$$w \leftarrow w - \eta \frac{\partial J}{\partial w} \qquad b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

- The gradient points uphill, so we go in the opposite direction.
- η is the **learning rate**: too small, it is slow; too large, it overshoots.
- J convex: gradient descent reaches the minimum.

What we need now: $\partial J / \partial w$ and $\partial J / \partial b$.



Step 1 · The derivative of the sigmoid

Write $\sigma(z) = (1 + e^{-z})^{-1}$ and differentiate (chain rule):

$$\sigma'(z) = -1 \cdot (1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1-\sigma(z)}$$

The second factor is $1 - \sigma(z)$, because $1 - \frac{1}{1+e^{-z}} = \frac{1+e^{-z}-1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$. So:

$$\sigma'(z) = \sigma(z)\,\big(1 - \sigma(z)\big) \qquad \text{or, with } a = \sigma(z): \quad \frac{\partial a}{\partial z} = a\,(1 - a)$$

Maximum at $z = 0$: $\sigma'(0) = 0.5 \times 0.5 = 0.25$. Keep this number in mind for session 5.

Step 2 · The gradient of the loss (chain rule)

One example: $z = w^\top x + b$, $a = \sigma(z)$, $\mathcal{L} = -[y \log a + (1-y)\log(1-a)]$. The chain $w_j \to z \to a \to \mathcal{L}$:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_j}$$

factor	computation
$\frac{\partial \mathcal{L}}{\partial a}$	$-\frac{y}{a} + \frac{1-y}{1-a} = \frac{-y(1-a)+(1-y)a}{a(1-a)} = \frac{a-y}{a(1-a)}$
$\frac{\partial a}{\partial z}$	$a(1-a)$ (step 1)
$\frac{\partial z}{\partial w_j}$, $\frac{\partial z}{\partial b}$	x_j and 1

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{a-y}{a(1-a)} \cdot a(1-a) \cdot x_j = (a-y)\,x_j \qquad \frac{\partial \mathcal{L}}{\partial b} = a - y$$

Step 3 · Vectorised over the dataset

J is the mean of the m losses, so its gradient is the mean of the m gradients:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)})\, x_j^{(i)}$$

With X of shape (m, n) (one example per row) and the vectors A, y of shape $(m,)$:

$$dw = \frac{1}{m}\, X^\top (A - y) \qquad db = \frac{1}{m}\sum_{i=1}^{m}(a^{(i)} - y^{(i)})$$

Check the shapes

$$\underbrace{X^\top}_{(n,\,m)}\ \underbrace{(A - y)}_{(m,)} = \underbrace{dw}_{(n,)}$$

To remember

The gradient is **error times input**: $(a - y)\, x$. Examples with a large error push the weights hardest. Same formula as linear regression, with $a = \sigma(z)$ instead of z.

Exercise 3 · One gradient-descent step by hand

	x_1	x_2	y
example 1	2	1	1
example 2	0	2	0

Start: $w = (0, 0)$, $b = 0$. Learning rate $\eta = 2$. Use $e^{-1.5} \approx 0.223$ and $e^{1} \approx 2.718$.

Exercise 12 min pen and paper, calculator

1. Forward: $a^{(1)}$, $a^{(2)}$ and the cost J.
2. Gradients: $dw = \frac{1}{2} X^\top (A - y)$ and db.
3. Update w and b.
4. New forward pass and new cost. Did J decrease?

Your notes / answers (saved in this browser)

Break: 10 minutes

Logistic regression in NumPy (1/2): the functions

```python
def initialisation(n):
    return np.random.randn(n), np.random.randn()        # w: (n,), b: a number

def model(X, w, b):
    return 1 / (1 + np.exp(-(X @ w + b)))               # A: (m,), the probabilities

def log_loss(A, y, eps=1e-15):                          # eps avoids log(0)
    return -np.mean(y * np.log(A + eps) + (1 - y) * np.log(1 - A + eps))

def gradients(A, X, y):
    return X.T @ (A - y) / len(y), np.mean(A - y)       # dw: (n,), db: a number

def update(dw, db, w, b, eta):
    return w - eta * dw, b - eta * db
```

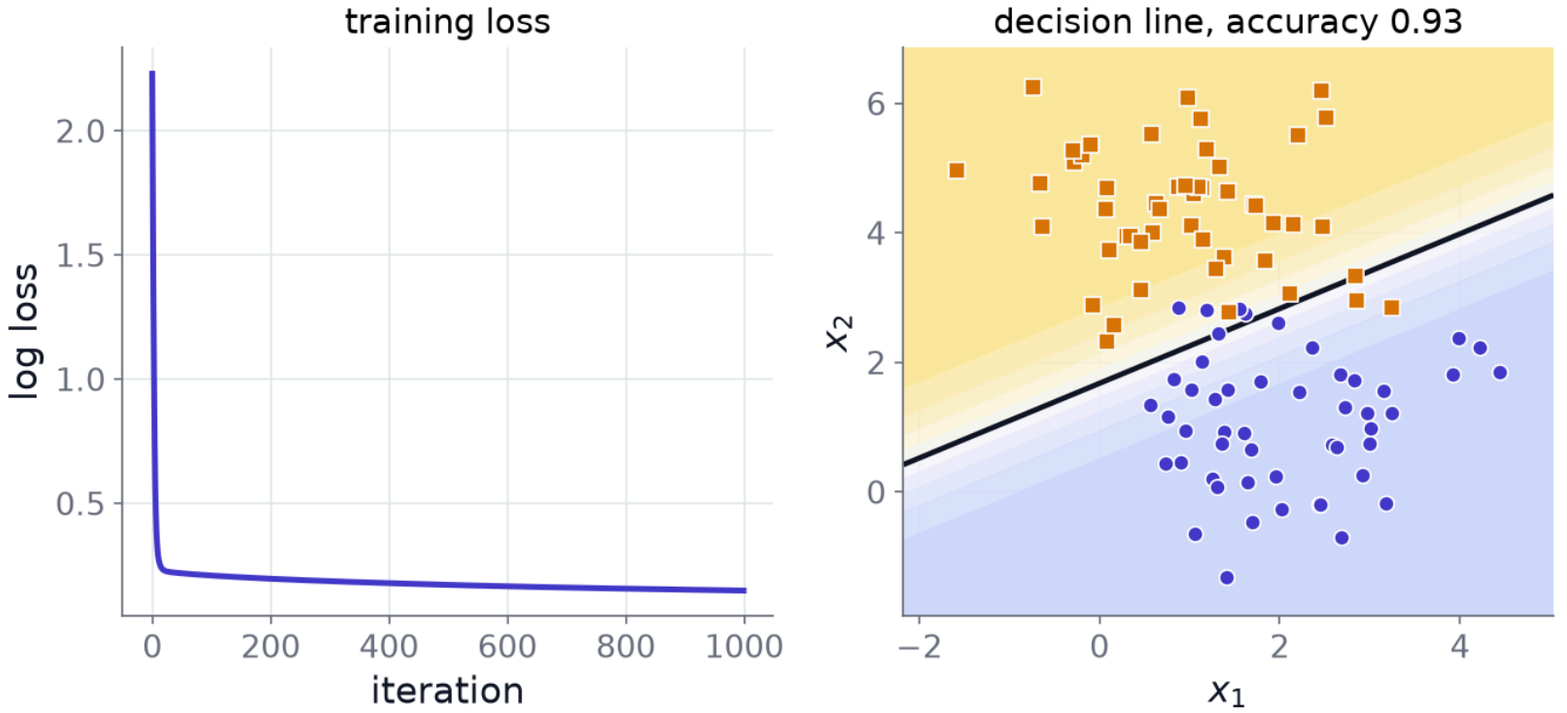
One function per formula of the course: forward $A = \sigma(Xw + b)$, cost J, gradients $dw = \frac{1}{m} X^\top (A - y)$, db, and the descent step.

Logistic regression in NumPy (2/2): the training loop

```
def logistic_regression(X, y, eta=0.1, n_iter=1000):
    w, b = initialisation(X.shape[1])
    losses = []
    for i in range(n_iter):
        A = model(X, w, b)                        # forward
        losses.append(log_loss(A, y))
        dw, db = gradients(A, X, y)               # gradients
        w, b = update(dw, db, w, b, eta)          # one step downhill
    return w, b, losses

X, y = make_blobs(n_samples=100, n_features=2, centers=2, random_state=0)
np.random.seed(0)
w, b, losses = logistic_regression(X, y)
print(losses[0], losses[-1], ((model(X, w, b) ⩾ 0.5) == y).mean())   # 2.229 0.148 0.93
```

The result: the loss decreases, the boundary is a line



Toxic plants (two features measured on each plant, class 1 = toxic): the loss falls from 2.229 to 0.148, accuracy 0.93. The colours show $a = P(y = 1 \mid x)$; the black line is $a = 0.5$, that is $w^\top x + b = 0$.

The same neuron in PyTorch: autograd

```
import torch
w = torch.tensor([2.0, -1.0], requires_grad=True)   # parameters: track their gradients
b = torch.tensor(-1.0, requires_grad=True)
x, y = torch.tensor([1.0, 3.0]), torch.tensor(1.0)
a = torch.sigmoid(w @ x + b)                        # forward pass: a = 0.1192
loss = -(y * torch.log(a) + (1 - y) * torch.log(1 - a))
loss.backward()                                     # autograd fills w.grad and b.grad
print(w.grad, b.grad)                               # tensor([-0.8808, -2.6424]) tensor(-0.8808)
```

- `requires_grad=True`: PyTorch records every operation done with this tensor.
- `loss.backward()` applies the chain rule backwards and stores $\partial \mathcal{L}/\partial w$ in `w.grad`.
- By hand (exercise 1 with $y = 1$): $(a - y)\, x = (0.119 - 1) \times (1, 3) = (-0.881, -2.642)$. Same numbers.

Why PyTorch wants logits: `BCEWithLogitsLoss`

Computing $\sigma(z)$ then $\log$ breaks for large $|z|$: in float32, $\sigma(-200)$ rounds to 0 and $\log 0 = -\infty$.

Substitute $\log \sigma(z) = -\log(1 + e^{-z})$ and $\log(1 - \sigma(z)) = -z - \log(1 + e^{-z})$ in the loss:

$$\mathcal{L} = (1 - y)\, z + \log(1 + e^{-z})$$

PyTorch evaluates it in a stable form, without ever computing $\sigma(z)$ alone. So the model outputs the **logit** z, and the loss applies the sigmoid itself.

```
z, y = torch.tensor([-200.0]), torch.tensor([1.0])
a = torch.sigmoid(z)
print(a)                          # tensor([0.])
print(-torch.log(a))              # tensor([inf])
loss_fn = torch.nn.BCEWithLogitsLoss()
print(loss_fn(z, y))              # tensor(200.)
```

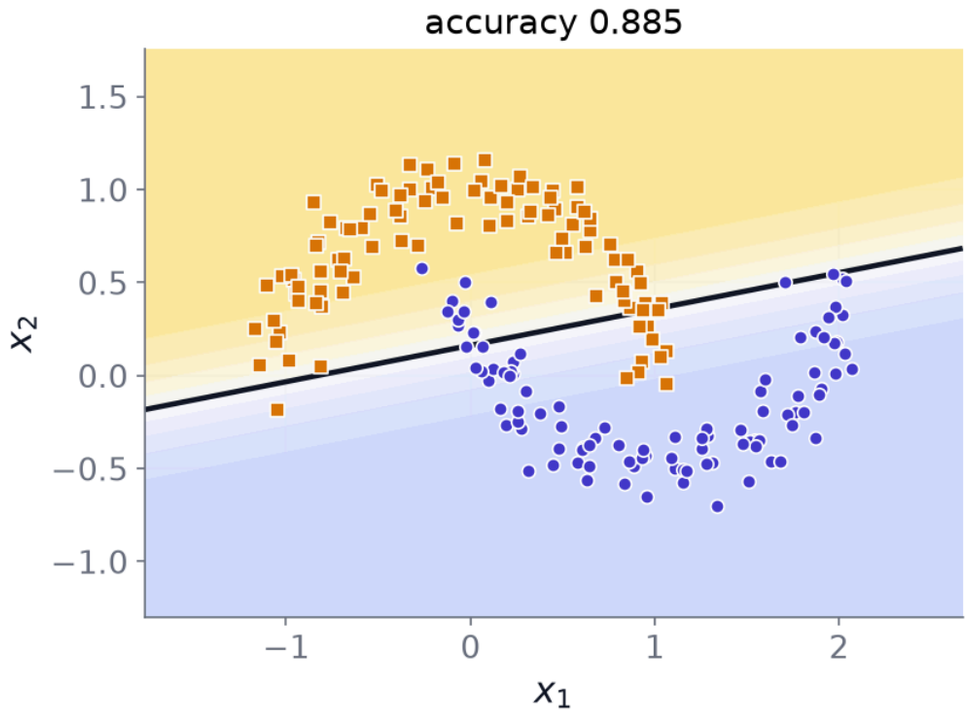
With $y = 1$: $\mathcal{L} = \log(1 + e^{200}) \approx 200$. Correct, and finite.

The idiomatic PyTorch version: the 5-line loop

```
import torch.nn as nn
X_t = torch.tensor(X, dtype=torch.float32)          # (m, 2)
y_t = torch.tensor(y, dtype=torch.float32)          # (m,)
torch.manual_seed(0)                                # same initial weights for everybody
model = nn.Linear(2, 1)                            # z = X @ W.T + b, W of shape (1, 2)
loss_fn = nn.BCEWithLogitsLoss()                    # sigmoid + log loss, stable
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for i in range(1000):
    optimizer.zero_grad()                           # 1. reset the gradients
    z = model(X_t).squeeze(1)                       # 2. forward: logits, shape (m,)
    loss = loss_fn(z, y_t)                          # 3. the log loss
    loss.backward()                                 # 4. gradients by autograd
    optimizer.step()                                # 5. w ← w - lr * grad
print(loss.item())                                  # 0.156 (accuracy 0.92)
```

The same algorithm as our NumPy code. Session 4 explains every PyTorch object in detail; today, just read the loop.

The limit: still a straight line



- The boundary $w^\top x + b = 0$ is a line, whatever the training.
- On the two moons the best line still misclassifies the tips: accuracy about 0.88.
- XOR is still impossible.

Next session

Several neurons, each drawing its own line, and a last neuron that combines them: the first **neural network**, trained by **backpropagation**.

TP · Logistic regression, from NumPy to PyTorch

TP 78 min tp/TP02_logistic_regression.ipynb

1. Part A (35 min): sigmoid, model, log loss, gradients, update, training loop, decision line in NumPy.
2. Part B (10 min): check the gradient formula with finite differences.
3. Part C (25 min): the same model in PyTorch, first with autograd by hand, then with `nn.Linear`.
4. Part D (8 min): the limit on the moons.

Deliverable: every cell prints "Exercise validated!".

Your notes / answers (saved in this browser)

Watch it on YouTube Machine Learnia, Deep Learning series

Today's maths and code, step by step.

Video 2
Le perceptron et la régression logistique
The perceptron and logistic regression

Video 3
Les gradients calculés en détail
The gradients, computed in detail

Video 4
Un rappel sur la vectorisation
A reminder on vectorisation

Video 5
Développer le tout avec NumPy
Coding it all in NumPy

Video 6
Application : chiens et chats
Application: cats versus dogs

Playlist
The whole Deep Learning series
10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. Sigmoid neuron = logistic regression: $a = \sigma(w^\top x + b) = P(y = 1 \mid x)$; the boundary is still a line.
2. Log loss = negative mean log-likelihood: $J = -\frac{1}{m}\sum[y \log a + (1 - y)\log(1 - a)]$.
3. Gradient: $\sigma' = \sigma(1 - \sigma)$, then $dw = \frac{1}{m} X^\top (A - y)$, $db = \frac{1}{m}\sum(a - y)$.
4. PyTorch: autograd computes the same gradient; `nn.Linear` + `BCEWithLogitsLoss` + `SGD` and the 5-line loop.

Next session: stack neurons into a network, solve XOR, and derive backpropagation.